

Boundary Case: LLM Consultation Timeout as Standalone Architectural Treatment in the Coordination Knowledge Substrate Pattern

Author: Wenxin Li (Independent Researcher)

ORCID: 0009-0004-8065-3235

Date: May 7, 2026

License: Creative Commons Attribution 4.0 International (CC BY 4.0)

Attribution

This work derives from and formalizes the Coordination Knowledge Substrate (CKS) pattern introduced in *Coordination Outside the Model: A Human-Governed Substrate Pattern for AI Systems* (Li, April 2026). It does not introduce new axioms. Its sole contribution is to formalize one boundary case in the CKS architecture — the **LLM consultation timeout boundary** — in which a consultation within a cell, performed under the AI-as-substrate-mediator commitment, fails to return a successful result. The note articulates how the architecture handles such failures, identifies the commitments the boundary stresses, names the legitimate architectural treatment, and identifies anti-pattern treatments that violate the architecture.

Abstract

LLM consultations performed within cells under the AI-as-substrate-mediator commitment do not always succeed. Consultations time out; vendor APIs return errors; outputs come back malformed; rate limits trigger; transient network failures interrupt the call. These events are operationally common, and the architectural treatment they require is non-obvious. A failed consultation still must not corrupt the substrate, must not break cell-behavior determinism, and must not bypass the orchestration rule under which it was performed. This note formalizes the consultation timeout boundary as a standalone architectural treatment with four components: the failure is recorded as a bounded-non-determinism event with full provenance, in the same way successful consultations are; the orchestration rule must itself specify how the cell behaves on failure; cell behavior under failure is deterministic relative to substrate state, the rule, and the recorded outcome; replay reproduces cell behavior using the recorded failure as the recorded outcome. The note identifies seven anti-pattern treatments that violate the architecture, articulates operational implications, and defines the boundary's scope relative to other failure boundary cases formalized separately.

1. Why the boundary case needs to be formalized as standalone

The CKS pattern's AI-as-substrate-mediator commitment names five properties the LLM must satisfy for every operation. Property B — that the LLM writes to substrate content only under orchestration rules authored by humans — assumes, on a default reading, that the LLM produces successful outputs the cell can consume. The default reading is sufficient for the architectural argument the source paper makes, but it does not address the failure path: a consultation that times out, returns a vendor error,

returns text that fails to parse against the form the rule expects, hits a rate limit, drops mid-call, or returns a refusal. Each is operationally common in production, and each leaves the cell in a state where the consultation has not produced a result the rule's success path can consume. Without a standalone treatment of this boundary, deployments are free to handle failures through ad-hoc patterns — vendor-managed retries, silent fallbacks, exception propagation that bypasses the rule — each of which violates one or more of the source paper's commitments. Naming the boundary as a standalone treatment gives downstream implementers a single reference for what failure handling must satisfy and what it must not do.

This note is the fourth in the boundary-case sequence. It addresses LLM consultation failure within a cell specifically, scoped distinct from vendor-level unavailability, composition-partner unavailability, and substrate-level failures, which are formalized separately.

2. The boundary case scenario

A cell consults an LLM under an orchestration rule, in accordance with Property B. The consultation fails — timeout, vendor HTTP error, malformed output that cannot be parsed against the schema the rule specifies, rate-limit rejection, authentication or authorization failure, mid-call network drop, model refusal that is not the answer the rule asked for. Each is a failure for the cell's purposes, because the consultation has not produced an output the rule's success path can consume.

The non-obviousness of the boundary lies in the temptation to treat consultation failures as engineering exceptions handled by the harness or the vendor SDK, outside the architectural scope. That treatment loses the commitments the architecture has made — substrate non-corruption, cell-behavior determinism, recorded provenance, the mediator role preserved — because exception handling outside the architecture is not subject to the same provenance, determinism, and mediation constraints. The architectural treatment is to bring failures inside the architecture as substrate-recordable events under rule-specified handling, rather than delegate them to a layer outside it.

3. Which architectural commitments are stressed

The boundary stresses five commitments simultaneously.

The mediator role under failure conditions. The five-property mediator definition assumes the LLM produces substrate-relevant content. Under failure it produces nothing the success path can consume, but the role must still hold: the LLM must not be authoritative over the substrate, and any substrate writes must occur under orchestration rules.

Rules specifying failure handling. Property B's success-path reading requires that LLM writes occur under human-authored rules. Its failure-path reading extends the requirement: rules must address what happens on failure, or cell behavior on the failure path is not rule-determined. Silence on the failure path is not the same as a rule.

Failures as a bounded-non-determinism category. The CKS pattern admits non-determinism only at specified categories, with LLM outputs recorded with attribution as the canonical case. Consultation failures extend this category — a failure is a non-deterministic outcome of an LLM call, recognized as such and recorded with attribution.

Bounded non-determinism within the mediator. The mediator role permits the LLM to be non-deterministic, but the non-determinism is bounded — it does not propagate to substrate writes outside the rule-mediated path. The bounding mechanism — recording, provenance, rule-mediated handling — must apply on failure the same way it applies on success.

Determinism under failure. The determinism contract requires that cell behavior be deterministic relative to substrate state plus rule plus recorded inputs. Under consultation failure, the recorded input includes the recorded failure. The contract does not require that the failure not occur; it requires that the cell's response be deterministic given the recorded failure as input.

4. The architectural treatment

The architectural treatment of the boundary has four components.

(a) The failure is recorded as a bounded-non-determinism event with full provenance. The failure is recorded in the substrate alongside the same provenance fields a successful consultation would carry: who attempted (cell, orchestration rule, cell execution identifier), when, what was requested, and what was returned (the failure indicator and any details — error code, timeout duration, malformed-output content, vendor response). The failure is not silently absent; it is present as a specific kind of recorded content, distinguishable from a successful consultation by outcome but otherwise of the same architectural type.

(b) The orchestration rule specifies how the cell behaves on consultation failure. The rule includes, as part of its specification, what the cell does on failure. The architecture admits a range of choices — bounded retry with each attempt recorded; fallback to a deterministic logic path the rule specifies; writing a "consultation-failed" record to the substrate as the cell's output; queueing the operation for human attention as a substrate-recordable artifact; rejecting the operation with a clear error state. The choice is the rule author's. What the architecture commits to is that the choice is made by the rule, in advance, and is itself substrate-resident authoritative content under the same human-governance commitments as any other rule.

(c) Cell behavior under failure is deterministic relative to substrate state, the rule, and the recorded outcome. Replaying the cell on the same substrate state, under the same rule (including its failure-handling specification), with the same recorded outcome, produces the same substrate writes. The recorded outcome is the recorded input the rule's deterministic handling consumes.

(d) Replay reproduces cell behavior using the recorded failure as the recorded outcome. A system replaying a cell that experienced a consultation failure does not re-attempt the consultation; it uses the recorded failure as the recorded LLM consultation outcome, the same way successful replays use the recorded successful output.

5. Anti-pattern treatments that violate the architecture

Seven anti-pattern treatments recur across systems that handle consultation failures without satisfying the architectural treatment. Each is named with the commitments it violates.

LLM-failure-causes-substrate-corruption. Uncontrolled substrate writes — partial outputs land in the substrate, malformed text is parsed and written anyway, exception state leaks into substrate fields. Violates Property B and the determinism contract.

Silent fallback. The cell falls back to a default value and the failure is not recorded. Replay produces different results because the recorded inputs do not include the failure that triggered the fallback. Violates the determinism contract and path-retraceability.

Auto-retry without recording. The vendor SDK or harness retries automatically and only the final attempt is exposed; the retry sequence is not recorded. Replay determinism breaks because the recorded outcome captures only the final attempt. Distinct from rule-specified retry, which is legitimate when the rule directs the cell to retry up to a bounded count and each attempt is recorded with provenance.

LLM-failure-bypasses-rule. Cell behavior on failure is not determined by the rule; the harness, runtime, or cell implementation handles it in some way the rule does not specify. Violates Property B's failure-path reading — the cell is operating outside the rule at the operational moment the rule must cover.

Vendor-managed retry without substrate record. A specialization of auto-retry-without-recording in which the vendor's infrastructure performs retries transparently and the cell never sees the intermediate attempts. Vendor logs are not substrate content; the substrate cannot be replayed from them.

Failure-as-success. The LLM returns an error or malformed response and the cell treats it as a successful consultation, parsing the error into the form the rule expected. Violates Property E's attribution and the determinism contract.

LLM-output-direct-to-substrate. The consultation's return value flows directly into substrate writes without rule-mediated handling. Property B is violated: the LLM is the author, not the rule. This is the canonical instantiation of the LLM-as-terminal-producer anti-pattern formalized separately, and the consultation timeout boundary is one of its most common operational settings, because failure handling is the place where rule mediation is most often skipped.

6. Operational implications

Four implications follow.

Rules must specify failure handling. A rule that covers only successful-output consumption is incomplete. Implementing the architecture means writing rules that cover the failure path explicitly — timeout, malformed output, vendor error, rate limit, each failure category the deployment expects. Treating failure handling as an engineering concern outside the rule leaves the architecture's commitments unbacked at the operational moment they matter most.

Retries must be recorded. Rule-specified retry is admissible only when each attempt is recorded as a bounded-non-determinism event with provenance. The recording requirement is what allows replay to reproduce cell behavior; vendor or harness retry logic that operates outside this recording is not retry the architecture admits.

Failure may itself be a substrate write. "Writing a consultation-failed record to the substrate as the cell's output" is a legitimate handling. Some settings benefit directly: the cell's output is the recorded fact that the consultation failed, and downstream cells consume that fact as input. The substrate carries the failure as content, not as an absence.

Replay uses recorded failure events. Systems that replay cells for audit, regression testing, or recovery use the recorded failure as the recorded outcome rather than re-attempting the consultation. Re-attempting on replay would defeat the determinism contract by introducing a new non-deterministic event.

7. Limits of the architectural treatment

The boundary applies specifically to LLM consultation failures within cells. Three adjacent failure categories are formalized separately.

Vendor-level LLM unavailability. A failure on a particular call is the boundary this note addresses. A sustained vendor outage that prevents any consultation from succeeding for an extended period raises additional questions — graceful degradation across many cells, whether the rule's bounded retry policy is meaningfully exercisable, posture under partial unavailability — formalized separately.

Composition partner unavailability. When a CKS substrate composes with another substrate or service, the failure of the partner during a cross-substrate operation is a different boundary case with different stressed commitments and a different architectural treatment, formalized separately.

Substrate-level failures. Failures of the substrate infrastructure itself — storage outages, schema corruption, persistence-layer errors — are not within scope and are addressed by their own formalizations.

The treatment in this note is not a guarantee of availability or recovery; it is a guarantee that, given the rule's failure-handling specification, the architecture's commitments hold across the failure event.

8. The architectural test

A system handles the consultation timeout boundary in a CKS-coherent manner if and only if all of the following hold at all times during the substrate's existence:

1. Every orchestration rule under which a cell consults an LLM specifies, as part of the rule's content, what the cell does when the consultation fails.
2. Consultation failures are recorded in the substrate as bounded-non-determinism events with the same provenance fields successful consultations carry.
3. Cell behavior on the failure path is deterministic relative to substrate state, the rule (including its failure-handling specification), and the recorded outcome.
4. Replay of a cell that experienced a consultation failure uses the recorded failure as the recorded outcome and produces the same substrate writes on every replay.
5. No substrate writes occur on the failure path outside the rule-mediated handling — neither from vendor-managed retry that is not recorded, nor from silent fallback, nor from harness-level

exception handling that bypasses the rule.

A system that fails any of (1)–(5) may handle consultation failures robustly by some other standard, but does not handle them in a CKS-coherent way.

9. Why naming this boundary as standalone matters

LLM consultation failures are not edge cases in production; they are routine events any non-trivial CKS deployment will encounter at frequency. Without an architectural treatment of the failure path, the patterns that emerge violate the architecture quietly: silent fallbacks pass tests; vendor-managed retries appear to deliver reliability; failure-as-success keeps cells from blocking. Each carries the architecture for the success path and breaks it for the failure path, with the breakage invisible until a replay diverges, an audit cannot reconstruct what happened, or a downstream system inherits substrate content the architecture's provenance commitments cannot account for. Naming the consultation timeout as a standalone boundary case gives downstream implementers a single architectural reference for what the failure path requires, and the boundary is detectable by the operational tests the architecture already commits to — replay divergence, provenance gaps, missing recorded events — when those tests are exercised against actual failure scenarios.

This note is the fourth in the boundary-case sequence. The next note formalizes the composition partner failure boundary, which carries this note's pattern — failure as bounded-non-determinism, rule-specified handling, deterministic-given-recorded-outcome — across the substrate boundary into multi-substrate composition. Subsequent work that handles LLM consultation failures should use the architectural treatment formalized here as the standalone reference; work that handles failures through vendor-managed retry, silent fallback, or harness-level exception handling without substrate recording is using a different treatment, and the difference should be named.

Source paper

Li, W. (2026). *Coordination Outside the Model: A Human-Governed Substrate Pattern for AI Systems*. April 2026. ORCID: 0009-0004-8065-3235.

How to cite this note

Li, W. (2026). *Boundary Case: LLM Consultation Timeout as Standalone Architectural Treatment in the Coordination Knowledge Substrate Pattern*. May 7, 2026. ORCID: 0009-0004-8065-3235.